

도착 시각(release time)과 soft due date를 갖는 입출고 병행 트럭 크로스도킹 스케줄링: 정확해 검증을 동반한 병목 유도 반복 지역탐색

김영수* 권상진†

연세대학교 인공지능학과, 서울

Abstract

입고와 출고를 병행하는 트럭(compound truck, 이하 병행 트럭)이 부분 하역된 뒤 특정 목적지의 출고 트럭으로 전환되는 다중 도어 크로스도킹 스케줄링 문제를 다룬다. 기존 병행 트럭 모델은 모든 트럭이 시각 0에 도착한다고 가정하고 총 작업완료시간(Makespan)만 최소화한다. 본 연구는 트럭별 도착 시각(Release Time)과, 위반을 허용하되 지연 비용을 부과하는 soft due date를 도입하고, 총 작업완료시간과 총 납기지연(Tardiness)의 합을 최소화하는 문제로 확장한다. 문제를 mixed-integer program과 CP-SAT 모형으로 정식화하고, 서로 겹치지 않는 학습·조정·시험 seed 집합을 사용한 재현 가능한 실험 프로토콜을 제시한다. 제안 방법인 VAA-GILS는 Vogel식 구성 기법으로 생성한 초기해에 best-improvement descent, 병목(bottleneck) 유도 연산자, 그리고 kick restart를 결합한 iterated local search이다. CP-SAT가 기준해를 제공한 실험 조건에서 VAA-GILS는 그 기준해 대비 평균 0.11–0.21%의 목적값을 1초 미만에 얻었다(CP-SAT는 76–237초). 이 가운데 소형 무제약 조건은 CP-SAT가 최적성을 증명한 경우로, 제안 방법은 증명된 최적해 대비 평균 0.21%였다. CP-SAT가 600초 이내에 실행 가능한 해를 찾지 못한 대형 시간 제약 조건에서도, 제안 방법은 비교 방법 가운데 가장 우수한 해를 제공하였다. Ablation 실험 결과, 성능 향상에는 descent, 병목 유도 연산자, restart로 구성된 결정론적 탐색 구조가 주요하게 기여한 것으로 나타났다. 한편 학습 기반 연산자 선택은 동일한 반복 횟수의 균등 선택 대비 추가 이득을 보이지 않았다.

주제어: cross-docking; compound truck; 도착 시각; soft due date; iterated local search; constraint programming; guided local search

1. 서론

크로스도킹은 입고 화물을 창고에 보관하지 않고 곧바로 출고트럭으로 환적하여 재고와 리드타임을 줄이는 물류 방식이다 [3, 13]. Joo와 Kim [4]이 도입하고 Shahmardan과 Sajadieh [12]가 발전시킨 병행 트럭 변형에서는 한 대의 트럭이 두 역할을 겸한다. 여러 목적지 화물을 싣고 도착한 뒤 한 목적지의 화물은 유지하고 나머지만 부분적으로 하역하며, 이후 해당 목적지의 나머지 화물을 적재해 출고 운송 트럭으로 출발한다. 부분 하역은 전량 하역 대비 총 작업완료시간을 크게 줄이는 것으로 보고되어 LTL 통합 네트워크에서 실용적 매력이크다.

기존 병행 트럭 모델의 두 가정이 현실성을 제약한다. 첫째, 모든 트럭이 시각 0에 가용하다고 가정하지만 실제 도크에서는 트럭마다 예약 도착 시각이 다르다. 둘째, 목적식이 순수 총 작업완료시간이지만 출고트럭의 출발은 위반 시 비용이 발생하는 due date에 묶여 있다.

도착 시각, 시간창(time window), 납기지연은 일반 크로스도킹 스케줄링에서 이미 연구되어 왔다 [14, 1, 2, 10, 11, 9]. 그러나 병행 트럭 설정은 temporal constraint의 의미를 바꾼다. 병행 트럭은 순수 입고트럭도 출고트럭도 아니다. 부분 하역 후 한 목적지의 화물을 유지하고, 해당 목적지로 들어오는 이송 화물을 기다린 뒤 비로소 출고 운송 트럭으로 출발한다. 이 부분 하역 변환은 하나의 precedence chain — 도착 → 부분 하역 → 유지 화물 → 이송 대기 → 출고 — 을 만들고, 이 chain은 트럭들을 공유 목적지를 통해

*제1저자.

†교신저자. Email: [지도교수 이메일 기입].

결합한다. 따라서 도착 시각을 추가한다고 해서 한 트럭의 시작 시각만 늦어지는 것은 아니다. 늦은 도착은 해당 트럭의 이송 화물에 의존하는 모든 출고 운송 트럭으로 전파되고, soft due date는 이 연쇄 지연을 납기지연 비용으로 바꾼다. 본 연구의 기여는 이 상호작용 — 알려진 도착 시각과 soft due date가 부분 하역 후 출고 운송 트럭으로 전환되는 과정과 결합되는 방식 — 을 정식화하고 분석하는 것이며, 저자가 아는 한 이 결합은 이전에 모형화된 바 없다.

본 논문의 기여는 세 가지다. 첫째, 부분 하역 병행 트럭 스케줄링을 트럭별 도착 시각과 soft due date를 갖는 문제로 확장하고, 이를 독립적인 스케줄 평가기와 일치하는 big- M MILP 및 reified CP-SAT로 정식화한다. 둘째, 기존 VAA construction과 기본 이웃 구조에 full best-improvement descent, 병목 유도 연산자, kick restart를 더한 VAA-GILS를 제안한다. 셋째, CP-SAT 기준해와 기존 방법과의 비교실험을 통해 제안 방법의 해 품질과 계산 시간을 검증한다. 이와 별도로, 학습 기반 연산자 선택의 추가 효과는 동일한 반복 횟수의 균등 선택과 비교하는 보조적인 ablation으로 검토한다.

본 논문의 구성은 다음과 같다. 제 2장에서 관련 연구를 정리하고, 제 3장에서 문제를 정의하며 혼합정수 계획 모형과 CP-SAT 모형을 제시한다. 제 4장에서 제안 알고리즘인 VAA-GILS를 서술하고, 제 5장과 제 6장에서 실험 설계와 결과를 제시한다. 제 7장에서 결과를 논의하고 제 8장에서 결론을 맺는다.

2. 관련 연구

시간 제약이 있는 크로스도킹 스케줄링. Boysen과 Flidner [3]는 도크 스케줄링 문제를 분류하였고, Van Belle 등 [13]은 분야를 조망하였다. 여러 결정론적 모형이 기존 입고·출고트럭에 대해 시간 제약을 이미 반영한다. Van Belle 등 [14]은 다중 도어·time window·납기지연을, Assadi와 Bagheri [1]는 ready time과 outbound 조기완료와 납기지연(earliness/납기지연)을, Bodnar 등 [2]은 time window와 mixed-service door를 다룬다. Molavi 등 [10]은 hard due date를, Rijal 등 [11]은 truck scheduling과 door assignment의 통합을 다룬다. 이들은 release/due date가 새롭지 않음을 보이나, 입고·출고트럭을 구분하며 부분 하역 후 출고 운송 트럭으로 전환되는 트럭을 모형화하지 않는다.

불확실·불완전 도착 정보. 도착 불확실성은 별도의 흐름이다. Konur와 Golias는 bounded arrival window [5]와 unknown arrival 하의 cost-stable schedule [6]을 다룬다. Larbi 등 [8]은 도착 정보의 완전·부분·부재(full/partial/absent) 정보를 구분하고, Ladier와 Alpan [7]은 time window 하의 robust schedule을, Xi 등 [17]은 two-stage conflict robust optimization을 제시한다. 본 모형은 결정론적이며 각 도착 시각은 스케줄 구성 시점에 알려져 있다. 따라서 이들 불확실성 모형은 본 모형을 직접 대체하기보다 향후 자연스럽게 확장할 수 있는 연구 방향에 해당한다.

가장 유사한 문제와 학습형 메타휴리스틱. Li 등 [9]은 무인 배송센터(distribution center)를 대상으로 mixed service mode dock의 통합 트럭 배정·스케줄링을 Q-learning 기반 ALNS로 푼다. 이들 역시 도착 시각과 연장 가능한 출발 예정 시각을 시간창(r_i, d_i)로 두고 납기지연과 총 작업완료시간을 목적에 포함하므로, 시간 제약의 형태 자체는 본 연구와 유사하다. 그러나 유연성이 도어 수준에 있어 도어가 적재 전용·하역 전용·혼합 모드 중 하나를 선택하고 트럭은 순수 입고·출고트럭으로 고정되며, 화물은 도어 간 직접 이송이 아니라 저장 구역을 경유해 AGV가 운반한다. 반면 본 문제의 유연성은 트럭 수준에 있어, 부분 하역된 병행 트럭이 특정 목적지의 운송 트럭으로 전환되고 화물은 도어 간에 직접 이송된다. 한편 이들을 포함하여 학습 기반 연산자 선택이 메타휴리스틱을 개선한다는 보고가 많으나, 본 연구는 강한 지역 탐색을 갖춘 뒤에는 학습 기반 선택 정책의 이득이 작다는 통제된 반례를 더한다.

3. 문제 정의

3.1 기본 모델

I, F, D, K, M 을 각각 병행 트럭, 출고트럭, 목적지, 상품 종류, 도어의 집합이라 하고 $|I| + |F| = |D|$, $|I| \leq |M|$ 이라 하자. 병행 트럭 i 는 목적지 d 행 상품 k 를 화물량 f_{idk} 만큼 실는다. 단위 처리시간 t_k , 도어 m, n 간 배치 이송시간 t_{mn} , 트럭 i 의 진입·출차 전환시간 DE_i, DL_i 에 대해 $h_{id} = \sum_k f_{idk} t_k$, $H_i = \sum_d h_{id}$

로 둔다. 각 병행 트럭은 유지할 목적지 하나와 도어 하나를(도어당 병행 트럭 최대 1대), 각 출고트럭은 남은 목적지 하나와 도어를 고르며, 같은 도어의 출고트럭은 작업 순서를 정한다. 각 목적지에는 정확히 하나의 운송 트럭이 배정된다.

각 트럭 $q \in I \cup F$ 의 도착 시각을 r_q 로 나타낸다. 따라서 r_i 는 병행 트럭 $i \in I$ 가 도어 작업을 시작할 수 있는 가장 이른 시각이고, r_f 는 출고트럭 $f \in F$ 가 도어 작업을 시작할 수 있는 가장 이른 시각이다. 위반이 허용되는 soft due date $\bar{d}_q$ 와 그에 따른 납기 지연 T_q 는 3.3절에서 정의한다.

시간 계산은 스케줄 평가기 규칙을 따른다. 병행 트럭 i (목적지 d 유지)의 하역은 $r_i + DE_i + \sum_{d' \neq d} h_{id'}$ 에 끝난다. 운송 트럭은 화물을 보내는 모든 트럭의 하역과 도어 간 이송이 끝나야 적재할 수 있다. 출고 운송 트럭 f 는 추가로 자신의 release와 도어의 선행 점유가 끝나는 시점에 도어 작업을 시작하여 $S_f = \max(\text{door available, destination ready, } r_f)$ 이며, 이후 진입(DE_f), 적재, 출차(DL_f)를 차례로 수행하므로 완료 시각은 $S_f + DE_f + L_d + DL_f$ 이다. 즉 S_f 는 적재 시작이 아니라 도어 작업 시작 시각이고, 적재는 $S_f + DE_f$ 에 시작한다. 총 작업완료시간 $\tau = C_{\max}$ 는 최대 완료 시각이다.

3.2 Mixed-integer 정식화

이진 변수 x_{idm} 은 병행 트럭 $i \in I$ 를 유지 목적지 d ·도어 m 에, z_{fdm} 은 출고트럭 $f \in F$ 를 배정한다. $X_{im} = \sum_d x_{idm}$, $Z_{fm} = \sum_d z_{fdm}$, $Z_{fd} = \sum_m z_{fdm}$ 로 두고, b_{fgm} 은 도어 m 에서 f 가 g 에 선행하면 1이다. 연속 변수 $U_i, C_i, S_f, C_f, C_{\max}$ 는 각각 병행 트럭의 하역 완료, 병행 트럭의 도어 작업 완료, 출고 도어 작업 시작, 출고 완료, 총 작업완료시간이다. $L_{id} = \sum_{j \neq i} h_{jd}$, $L_d = \sum_i h_{id}$, B 는 유효한 horizon 상한이다.

배정 제약은 다음과 같다.

$$\sum_{d,m} x_{idm} = 1 \quad \forall i \quad (1)$$

$$\sum_{d,m} z_{fdm} = 1 \quad \forall f \quad (2)$$

$$\sum_{i,m} x_{idm} + \sum_{f,m} z_{fdm} = 1 \quad \forall d \quad (3)$$

$$\sum_{i,d} x_{idm} \leq 1 \quad \forall m \quad (4)$$

병행 트럭의 시간 관계와 유입 이송 선행 제약은 다음과 같다.

$$U_i = r_i + DE_i + H_i - \sum_{d,m} h_{id} x_{idm} \quad \forall i \quad (5)$$

$$C_i \geq U_i + L_{id} + DL_i - B(1 - x_{idm}) \quad \forall i, d, m \quad (6)$$

$$C_i \geq U_j + t_{nm} + L_{id} + DL_i - B(2 - x_{idm} - X_{jn}) \quad (7)$$

식 (7)은 모든 i, d, m, n 과 $\sum_k f_{jdk} > 0$ 인 기여 트럭 $j \neq i$ 에 대해 성립한다.

출고 운송 트럭에 대해서는 다음이 성립한다.

$$S_f \geq r_f \quad \forall f \quad (8)$$

$$C_f \geq S_f + DE_f + L_d + DL_f - B(1 - Z_{fd}) \quad \forall f, d \quad (9)$$

$$S_f \geq U_i + t_{nm} - B(2 - z_{fdm} - X_{in}) \quad (10)$$

$$S_f \geq C_i - B(2 - Z_{fm} - X_{im}) \quad \forall f, i, m \quad (11)$$

같은 도어 m 을 쓰는 출고트럭 쌍 $f < g$ 에 대한 순서 결정과 non-overlap 제약은 다음과 같다.

$$b_{fgm} + b_{gfm} \geq Z_{fm} + Z_{gm} - 1, \quad b_{fgm} \leq Z_{fm}, \quad b_{gfm} \leq Z_{gm} \quad (12)$$

$$S_g \geq C_f - B(1 - b_{fgm}) \quad (13)$$

$$S_f \geq C_g - B(1 - b_{gfm}) \quad (14)$$

끝으로 총 작업완료시간은 다음으로 정의된다.

$$C_{\max} \geq C_i \quad \forall i, \quad C_{\max} \geq C_f \quad \forall f \quad (15)$$

CP-SAT 모형은 big- M disjunction을 enforcement literal로 대체하고 모든 시간값을 가장 가까운 0.01 단위로 반올림한 뒤 100배하여 정수로 변환하며, 모든 배정을 고정하면 독립 evaluator의 완료 시각이 재현되어 model fidelity가 확인된다.

3.3 시간 제약 확장과 목적식

각 트럭 $q \in I \cup F$ 는 앞서 정의한 도착 시각 $r_q \geq 0$ 과 soft due date $\bar{d}_q$ 를 갖고 $T_q \geq \max\{0, C_q - \bar{d}_q\}$ 이다. 목적식은

$$\min J = C_{\max} + \lambda \sum_{q \in I \cup F} T_q, \quad \lambda = 1 \quad (16)$$

$r_q = 0, \bar{d}_q = \infty$ 이면 기존 총 작업완료시간 모델로 환원된다. CP-SAT는 incumbent와 증명된 하한을 함께 반환하며, 하한과 일치가 증명될 때에만 최적으로 부른다.

3.4 조합적 하한

CP-SAT가 제공하는 하한이 약한 실험 조건에서 격차를 보고하기 위해 도어 경합을 완화한 두 유효 하한을 사용한다. 병행 트럭 i (목적지 d 유지)의 최속 완료와 출고트럭 f 의 최속 완료는

$$EF_i = r_i + DE_i + \min_{d \in D} \left(H_i - h_{id} + \sum_{j \in I \setminus \{i\}} h_{jd} \right) + DL_i \quad (17)$$

$$EF_f = r_f + DE_f + \min_{d \in D} \sum_{j \in I} h_{jd} + DL_f \quad (18)$$

critical-chain 하한은 $LB_{cc} = \max_{q \in I \cup F} EF_q$ 이다. door-area 하한은 각 트럭의 최소 도어 점유

$$o_i = DE_i + \left(H_i - \max_d h_{id} \right) + \min_d \sum_{j \in I \setminus \{i\}} h_{jd} + DL_i \quad (19)$$

$$o_f = DE_f + \min_d \sum_{j \in I} h_{jd} + DL_f \quad (20)$$

을 도어 수로 나눈 값이다.

$$LB_{da} = \frac{1}{|M|} \left(\sum_i o_i + \sum_f o_f \right) \quad (21)$$

두 하한 모두 제약을 완화하여 얻으므로 이들의 최댓값 $LB_\tau = \max\{LB_{cc}, LB_{da}\}$ 는 총 작업완료시간의 유효 하한이다. 또한 각 EF_q 가 C_q 의 하한이므로 구조적 지연 하한은

$$LB_T = \sum_{q: \bar{d}_q < \infty} \max\{0, EF_q - \bar{d}_q\} \quad (22)$$

이며, $LB_J = LB_\tau + \lambda LB_T$ 는 식 (16)의 유효 하한이다.

4. 휴리스틱 알고리즘

VAA-GILS는 iterated local search의 구현체로, 연산자 내부 sampling과 acceptance를 제외하면 결정론적이다. 이하에서 원 모델은 Shahmardan과 Sajadieh [12]가 제안한 SA-RL5를 가리키며, VAA 구성 휴리스틱과 기본 이웃 7종은 이 모델에서 가져왔다.

초기해 구성. 원 모델의 VAA는 Vogel식 regret으로 병행 트럭의 유지 목적지를 정하고, 완료 시각 우선

순위에 따라 출고 목적지를 배정한다. 도어 배정은 먼저 처리되는 트럭을 도어 간 이송시간 합이 가장 작은 중앙 도어에 두고, 나머지 트럭은 작업 종료가 가장 이른 도어에 차례로 삽입한다. 모든 완료 예정 시각 계산에는 도착 시각을 반영한다.

Descent. 세 가지 이동 유형(출고트럭 재배치, 병행 트럭의 도어 교환/빈 도어 이동, 두 트럭의 목적지 교환)에 대한 best-improvement 지역탐색으로, 지역 최적해(Local Optimum)에서 종료하며 초기해·모든 신규 incumbent·최종해에 적용한다.

반복 탐색. 1,000회 반복한다. 선택 정책이 연산자 하나를 뽑아 현재 해에서 후보를 생성하고, 후보가 전역 최선해를 개선하면 descent를 적용해 incumbent를 갱신한다. acceptance는 초기 온도 $T_0 = \max\{1, 0.05 J(s_0)\}$, 기하 냉각 $T \leftarrow 0.995 T$ 의 simulated annealing이다.

Kick restart와 reheating. 30회 연속 개선이 없으면 **kick restart**를 수행한다. 현재 최선해 s^* 의 복사본에서 다시 시작하되, 아래 기본 이웃 7종에서 균등 무작위로 뽑은 이동(kick)을 3회 연속 적용한다. 각 이동은 직전 이동의 결과에 적용되며, 재시작 직후에는 descent를 수행하지 않는다. 이어서 $T \leftarrow \max\{T, 0.02 J(s^*)\}$ 로 온도를 reheat한다.

Operator pool. 기본 이웃 7종은 원 모델의 이웃 구조를 그대로 사용한다: (k1) 두 트럭의 목적지 교환, (k2) 병행 트럭 간 도어 교환, (k3) 출고트럭 간 도어 교환, (k4) 출고트럭을 다른 도어로 삽입, (k6) 하역시간 기반 룰렛 선택으로 병행 트럭을 도어에 삽입, (k7) 적재시간 기반 룰렛 선택으로 출고트럭을 도어에 삽입, (k8) 룰렛 선택으로 병행 트럭에 목적지를 배정. 원 모델은 이동을 k1–k8로 번호매기나 k5는 정의하지 않아 실제 구현은 7종이다. 여기에 현재 병목을 먼저 식별한 뒤 그 주변에서 가능한 후보를 모두 평가해 최선을 택하는 guided 연산자 4종을 더한다: (g1) critical door의 마지막 출고트럭을 최선 도어·위치로 재배치, (g2) critical 트럭의 목적지를 전 트럭과 대조해 최선 교환, (g3)/(g4) most tardy 트럭 기준의 동일 이동 (tardy인 트럭이 없으면 g1/g2로 fallback). 고속 증분 evaluator(평가당 수십 μs)가 descent와 guided 이동을 저렴하게 만든다.

선택 정책과 원 모델의 비교. 선택 정책은 교체할 수 있으며 기본값은 균등 무작위 선택이다. 학습 정책(SA-RL5의 tabular Q-learning [15], 규모 불변 특징(feature)으로 오프라인 학습한 transfer DQN [16])은 ablation 목적으로만 평가하며 제안 방법의 일부가 아니다. tabular Q는 원 모델과 같이 무개선 구간 5-상태에서 ϵ -greedy 탐험과 룰렛 활용을 쓰되, 보상은 원 모델의 이전 보상(후보가 현재 해보다 나쁘지 않으면 1, 아니면 0) 대신 신규 incumbent 2·비악화 1·그 외 0의 셰이핑 보상을 사용한다. 이는 학습 신호를 강화할 뿐이므로 비교 대상 선택 정책을 불리하게 만들지 않는다. 원 모델 SA-RL5는 RL이 단일 이동을 고르고 SA accept/reject로 한 단계가 끝나는 순수 simulated annealing인 반면, VAA-GILS는 그 한 연산자 적용 단계를 iterated local search 안에 넣고 guided 연산자, 개선 이동마다의 full descent, kick restart를 더한다. 가장 중요한 차이는 개선 이동 이후의 descent이며, ablation은 이것이 품질 이득의 대부분을 설명함을 보인다. 두 방법의 구조적 차이는 <Table eftab:base>에 정리하였다.

Table 1: Structural comparison of the base SA-RL5 and VAA-GILS.

항목	SA-RL5 (base)	VAA-GILS (제안)
전체 틀	순수 simulated annealing	iterated local search (SA는 acceptance만)
반복당 연산자	1개	1개 (동일)
연산자 집합	이웃 7종	그 7종 + guided 4종
연산자 선택	online Q-learning/roulette	균등 무작위 (학습은 ablation 전용)
개선 이동 이후	없음 (SA step이 전부)	full best-improvement descent
정책 처리	없음	kick restart + reheat
acceptance	SA(Metropolis)	SA(Metropolis; ablation상 생략 가능)
목적식	총 작업완료시간	총 작업완료시간 + λ ·납기 지연

5. 실험 설계

5.1 인스턴스 생성

병행 트럭 $|I|$, 출고트럭 $|F|$, 목적지 $|D| = |I| + |F|$, 도어 $|M| = |I|$ 로 두고 세 규모를 사용한다: S ($|I|, |F|$) = (6, 3), M (12, 6), L (20, 10). 따라서 ($|I|, |D|, |M|$)는 각각 (6, 9, 6), (12, 18, 12), (20, 30, 20)이고 병행 트럭 비율 $|I|/|D|$ 는 모두 2/3이다. S는 CP-SAT가 제한시간 안에 최적성을 증명할 수 있는 규모이고 L은 실행 가능해조차 얻기 어려운 규모여서 두 극단을 함께 본다. 병행 트럭 비율을 높게 잡은 것은, 이 비율이 낮으면 문제가 일반 크로스도킹 스케줄링에 가까워져 부분 하역 구조의 영향이 드러나지 않기 때문이다.

상품은 3종이며 트럭 $q \in I \cup F$ 에 대해 $f_{idk} \sim U[0, 20]$, $t_k \sim U[1, 4]$, $DE_q, DL_q \sim U[1, 5]$ 로 모두 정수 균등분포에서 뽑는다. 화물이 특정 목적지로 쏠리는 정도는 이와 별개의 조건이며, 보고하는 모든 실험은 쏠림이 없는 조건을 사용한다(쏠린 조건과 군집 조건은 부록 A의 transfer DQN 학습에만 쓴다). 도어는 $[0, 100]^2$ 에 균등 랜덤 좌표로 배치하고 도어 간 이송시간 t_{mn} 은 두 좌표 사이 유클리드 거리를 10으로 나눈 값이다. 도어를 일렬로 늘어놓으면 이송시간이 도어 번호 차이로 정해져 도어 배정이 해에 미치는 영향이 작아지므로, 배정 결정이 실제로 품질을 가르도록 무작위 배치를 쓴다. 거리를 10으로 나누는 것은 이송시간을 처리시간과 같은 크기 범위에 두기 위해서다.

5.2 시간 제약 수준

시간 제약은 none, medium, tight 세 수준이다. none은 $r_q = 0$, $\bar{d}_q = \infty$ 로 두어 모든 트럭이 시각 0에 가용하고 납기가 없는 기존 모델과 같으며, 본 연구의 확장을 꺾을 때의 대조군 역할을 한다. medium과 tight에서는 $r_q \sim U[0, \rho H]$, $\bar{d}_q = r_q + \delta H$, $(\rho, \delta) = (0.25, 0.60), (0.50, 0.35)$ 이다. 여기서 $H = 2 \sum_{i \in I} H_i / |M|$ 는 시간 제약이 없을 때의 총 작업완료시간 추정치로, 화물 한 단위가 하역과 적재로 두 번 처리되므로 계수 2를 둔다. 시간 제약을 H 의 비율로 주면 규모가 달라져도 압박 수준이 유지된다. ρ 는 도착이 퍼지는 폭, δ 는 도착 후 납기까지의 여유를 뜻하므로, tight는 두 방향으로 동시에 어려워지는 조건이다.

규모 3종과 시간 제약 3수준을 교차해 총 9개의 실험 조건을 만들고, 각 조건을 S-none, M-tight와 같이 규모-수준 형태로 표기한다.

5.3 비교 대상과 측정

비교 대상은 다섯이다. (i) VAA는 구성 휴리스틱만 적용한 초기해, (ii) Paper-SA-RL5는 원 모델의 재구현이며, (iii)–(v) GILS-uniform, GILS-tabular, GILS-DQN은 선택 정책만 바꾼 제안 방법의 세 변형이다. 제안 방법의 기본값은 GILS-uniform이고 나머지 두 변형은 학습 기반 선택의 효과를 보기 위한 것이다. 여기에 CP-SAT 결과를 함께 보고한다. 원 모델은 도착 시각과 납기를 다루지 않으므로 Paper-SA-RL5는 none 조건에서만 비교한다.

인스턴스 seed는 학습·조정·시험 세 집합으로 서로소가 되게 나눈다. 학습 정책은 학습 집합에서만 훈련하고 λ 와 같은 설계 값은 조정 집합에서만 고르므로 시험 집합의 성능이 부풀려지지 않는다. 주해품질 비교(<Table 2>)는 조건당 시험 인스턴스 20개를 각각 5회 반복 실행한다. 인스턴스마다 난이도가 다르므로 여러 개가 필요하고, 알고리즘에 무작위성이 있으므로 같은 인스턴스도 여러 번 실행한다. 통제된 ablation(<Table 3>–<Table 5>)은 같은 엔진에서 구성요소만 떼어내는 짝지은 비교여서 인스턴스 간 난이도 편차가 상쇄되므로 조건당 5개만 사용한다.

같은 인스턴스의 5회 반복은 서로 독립이 아니므로, 먼저 인스턴스별로 평균해 조건당 값 하나를 만든 뒤 짝지은 인스턴스 평균에 양측 Wilcoxon signed-rank test를 적용한다. 주 지표는 인스턴스별 격차 Δ_{bo} 이며, 해당 인스턴스에 대해 모든 방법과 모든 반복 횟수, CP-SAT 결과까지 통틀어 관측된 가장 좋은 목적값을 기준으로 삼는다. 대형 조건에서는 CP-SAT 기준해가 없어 최적해 대비 갭을 쓸 수 없기 때문이다.

GILS 계열의 종료 조건은 반복 횟수 1,000회로 통일한다. CP-SAT는 인스턴스당 1회, 8 thread로 S에 300초(5개), M과 L에 600초(조건당 2개)를 부여하며 S-none 5개에서만 최적성이 증명된다. 실험은 Apple M2 칩(8코어)과 16 GB 메모리를 갖춘 macOS 환경에서 Python 3.12와 OR-Tools 9.15로 수행하였다.

6. 실험 결과

6.1 해 품질

<Table eftab:main>에 20개 인스턴스에 대한 시험 결과를 요약한다. CP-SAT가 기준해를 제공한 실험 조건에서 GILS는 이에 근접하며, 기본 선택 정책인 균등 선택 기준 0.11–0.21%이다 (학습 선택 정책을 포함한 전체 변형은 0.1–0.6%). 같은 선택 정책 기준으로 GILS는 VAA를 평균 2.65%($n = 180, p < 10^{-4}$), SA-RL5를 1.38%($n = 60, p < 10^{-4}$) 능가한다. 실행 시간은 S/M/L에서 각각 0.11, 0.52, 2.3초로, 같은 조건의 CP-SAT(76, 237, 376초)보다 두세 자릿수 짧다.

6.2 핵심 구성요소 분석: 성능은 어디에서 오는가

세 가지 통제된 ablation(반복 횟수 1,000회 고정)으로 엔진을 분해한다.

Operator pool. 선택 정책을 균등 선택으로 고정하고 연산자 집합만 기본(7종)/핵심(+g1,g2)/전체(+g3,g4)로 바꾸면 Δ_{bo} 가 9개 실험 조건 전부에서 기본 $\geq$ 핵심 $\geq$ 전체로 단조 감소한다(<Figure 1>). 짝지은 Wilcoxon(<Table 3>)은 유도 연산자가 목적값을 일관되게 ·유의하게 낮춤을 보인다.

Table 3: Operator-pool ablation (5 instances $\times$ 5 replications per condition, the number of iterations fixed at 1,000; two-sided Wilcoxon, positive mean favors the richer operator set).

대비	n	개선(%p)	p
기본 $\rightarrow$ 핵심 (g1,g2 추가)	136	+0.114	$< 10^{-4}$
핵심 $\rightarrow$ 전체 (g3,g4 추가)	172	+0.048	$< 10^{-4}$
기본 $\rightarrow$ 전체 (유도 전부)	166	+0.161	$< 10^{-4}$

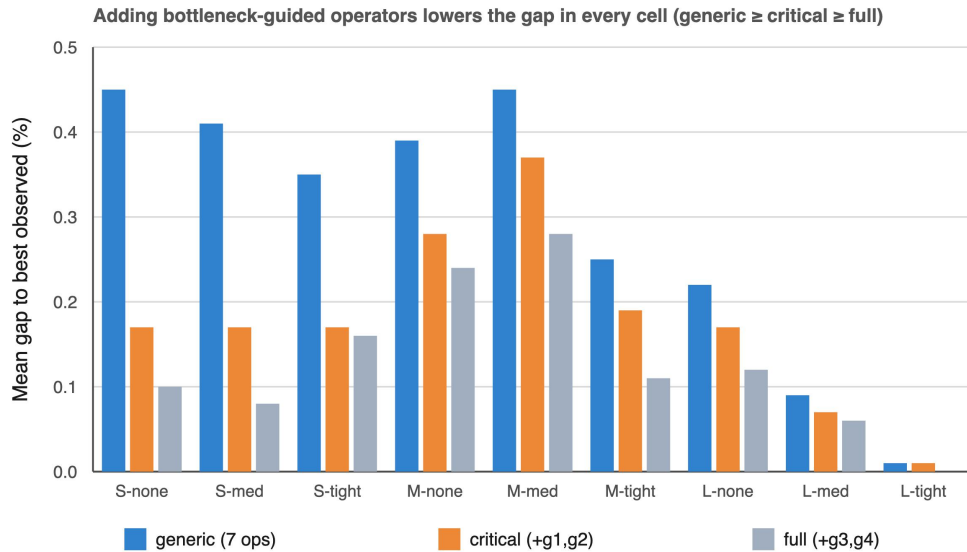


Figure 1: Mean gap to the best observed solution by operator set.

Engine components. 연산자 집합을 전체로, 선택 정책을 균등 선택으로 고정하고 구성요소를 하나씩 제거하면(<Table 4>), best-improvement descent의 기여가 가장 크고 kick restart가 그다음이며, VAA 초기화는 통계적으로 유의하지 않고 확률적 acceptance는 제거해도 목적값이 나빠지지 않는다(<Figure 2>).

Table 4: Engine-component ablation (5 instances $\times$ 5 replications per condition; leave-one-out, relative objective degradation when a component is removed).

제거 구성요소	n	악화(%p)	p
best-improvement descent	45	+0.156	$< 10^{-4}$
kick restart	43	+0.069	$< 10^{-4}$
VAA initial solution	45	+0.022	0.114
stochastic acceptance	45	-0.026	0.0014

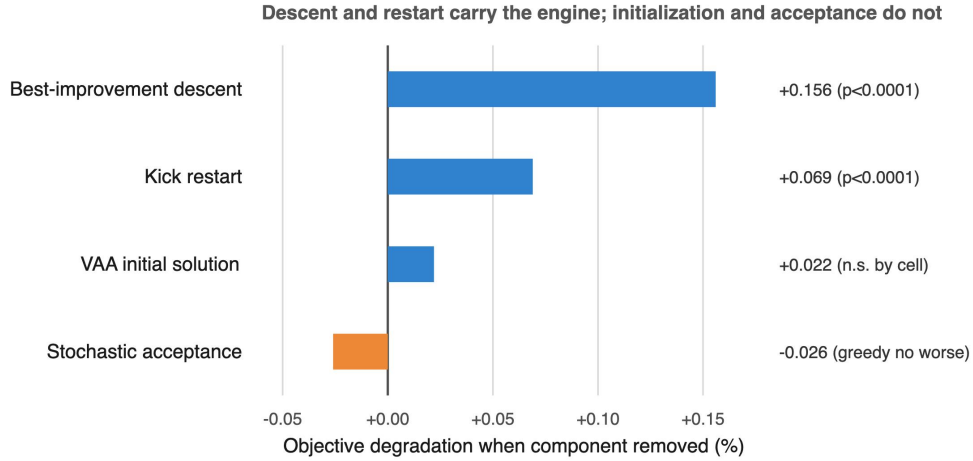


Figure 2: Objective degradation when each component is removed.

6.3 보조 분석: 학습 기반 연산자 선택

연산자 집합과 구성요소를 전체로 고정하고 선택 정책만 바꾸면(<Table 5>), 어떤 학습 정책도 균등 선택을 실용적으로 능가하지 못한다. 반복 횟수를 50–3,000회로 바꾸어도 결과는 같고 우열의 방향이 반복 횟수에 따라 뒤집히며(부록 <Figure 4>), 이러한 부정적 결과는 무제약 조건만으로도 성립하므로 시간 제약 확장 때문에 나타난 결과가 아니다.

Table 5: Effect of the selection policy (5 instances $\times$ 5 replications per condition, 1,000 iterations; two-sided Wilcoxon, positive mean favors the first method).

비교	평균차(%p)	p	판정
tabular vs uniform	-0.01	0.150	차이 없음
tabular vs DQN	+0.10	$< 10^{-4}$	DQN 열세
uniform vs DQN	+0.11	$< 10^{-4}$	DQN 열세

6.4 Tardiness-weight 민감도

식 (16)이 시간 단위 성분 둘을 합치므로 조정 집합에서 $\lambda \in \{0, 0.25, 0.5, 1, 2, 4, 8\}$ 을 스위칭한다. 물리적 총 작업완료시간과 총 납기지연을 인스턴스별로 min-max 정규화해 평균한다(<Figure 3>). 납기지연 감소의 대부분은 $\lambda = 0$ 에서 1 사이에서 일어나고 이후에는 그 폭이 줄어든다. 전체 스위칭 구간에서 평균 총 작업 완료시간은 약 1% 상승, 평균 납기지연은 2–3% 하락한다. $\lambda = 1$ 을 채택한다: 완만한 총 작업완료시간 비용으로 납기지연이 거의 최소에 이르고 두 시간 성분에 동일 가중을 준다. 이 스위칭은 시험이 아닌 조정 인스턴스를 사용한 설계 정당화이다.

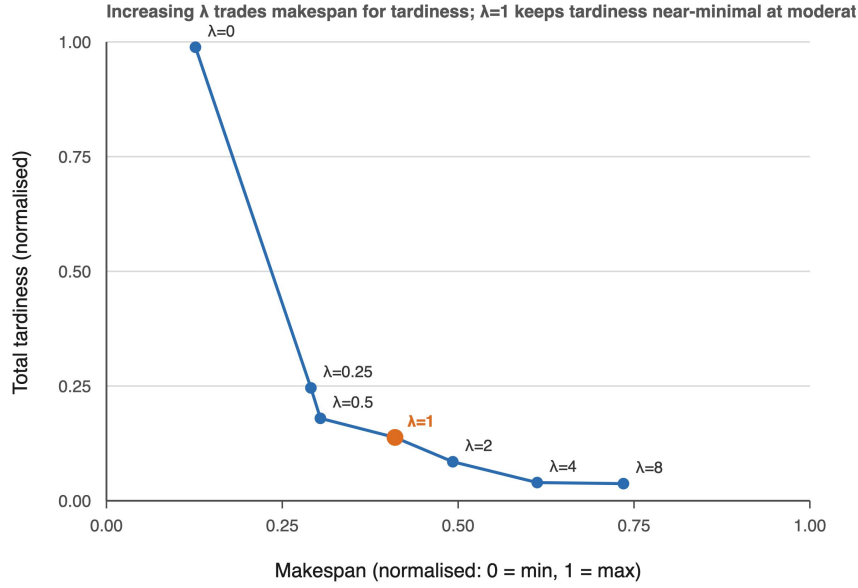


Figure 3: Normalized makespan-tardiness trade-off on tuning instances.

7. 논의

학습 기반 선택 정책이 균등 선택 대비 이득을 주지 못한 것은 학습 일반에 대한 결론이 아니라 본 논문의 탐색 구조가 낳은 결과다. 후보 하나의 평가에 수십 μs 밖에 들지 않아 연산자를 잘 고르는 것보다 많이 시도하는 편이 유리하고, 모든 신규 최선해 뒤에 결정론적 descent가 따라붙어 어떤 연산자로 그 지점에 이르렀는지의 차이를 상당 부분 흡수한다. 반복 횟수별 격차 곡선도 대부분의 시험 인스턴스에서 해 품질이 1,000회 이전에 평탄해짐을 보인다. SA acceptance와 reheating은 원 모델과 비교할 수 있도록 유지하였으나, ablation은 본 설정에서 이 둘이 해 품질에 필수적이지 않음을 보인다.

선택 정책이 예측할 대상이 마땅치 않다는 점도 작용한다. 본 문제의 인스턴스는 정적이고 모든 도착 시각이 최적화 이전에 알려져 있어, 실행 도중 새로 드러나 정책이 활용할 정보가 없다. 도착 정보가 시간에 따라 공개되는 동적 도크 운영이나 이동 하나의 평가가 비싼 시뮬레이션 기반 모형에서는 판단이 달라질 수 있다. 따라서 본 결과의 함의는 학습이 무용하다는 것이 아니라 비교 조건에 있다. 학습 기반 선택 정책은 동일한 엔진과 동일한 반복 횟수 아래 균등 선택과 견주어야 하며, 그 통제가 없으면 descent · restart · 강한 연산자 집합의 이득이 정책의 성과로 잘못 귀속될 수 있다.

8. 결론

본 연구는 부분 하역 병행 트럭 스케줄링에 도착 시각과 soft due date를 도입하고, MILP · CP-SAT 정식화와 유효 하한으로 휴리스틱 해를 검증하였다. CP-SAT 기준해가 있는 실험 조건에서 VAA-GILS의 목적 값은 해당 기준해 대비 평균 0.11–0.21%였고(소형 무제약 조건에서는 CP-SAT가 최적성을 증명하였다), 대형 시간 제약 조건에서는 수 초 안에 비교 방법 가운데 가장 좋은 해를 제공하였다. ablation은 성능이 best-improvement descent, guided 이동, restart라는 결정론적 구조에서 비롯되며, 학습 기반 연산자 선택 · 학습된 초기해 · 확률적 acceptance는 이 환경에서 실용적 이득을 주지 않음을 보였다. 향후에는 도착 정보가 실시간으로 공개되는 동적 문제, 대형 시간 제약 조건을 위한 더 강한 하한, 산업 데이터를 활용한 사례 연구가 필요하다.

A. Transfer DQN 구현 세부

transfer DQN은 매 반복 11개 연산자 집합에서 하나를 고른다. Q-network는 27차원 규모 불변 상태를 연산자별 Q값으로 사상하는 MLP로, 64 unit의 hidden layer 2개와 ReLU를 쓴다. 학습은 용량 100,000의 replay buffer를 사용하며, 500 transition warm-up 후 매 step 1회 gradient step으로 Huber(smooth- L_1) temporal-difference loss를 Adam (learning rate 10^{-3} , discount $\gamma = 0.9$)으로 최소화한다. mini-batch는 64, target network는 500 step마다 동기화한다. reward는 shaped signal(신규 incumbent 2, 비악화 1, 그 외 0)이다. agent는 학습 집합(S/M 규모, 모든 화물 분포·시간 제약 조건)에서 500회 실행으로 오프라인 학습(ϵ -greedy) 후, 시험 인스턴스에 zero-shot ($\epsilon = 0$, 갱신 없음)으로 적용한다. 결과가 균등 선택 대비 개선이 없다는 것이므로, 재현·강화가 가능하도록 network·학습 스크립트·seed를 공개한다.

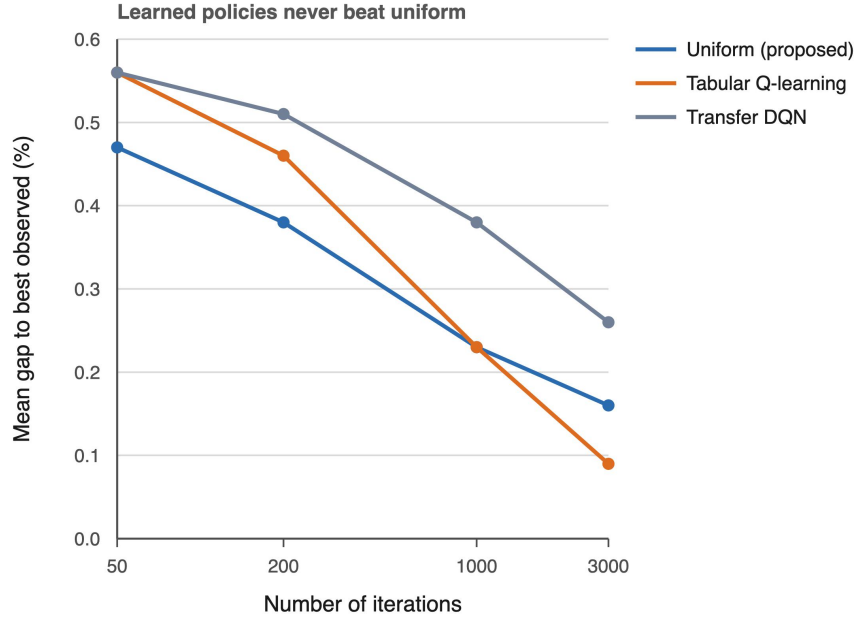


Figure 4: Mean gap to the best observed solution versus the number of iterations.

생성형 AI 사용 명시

원고 준비 과정에서 저자들은 언어 교정, 원고 구성, 참고문헌 확인, 계산 코드 개발·검토 보조를 위해 OpenAI Codex(GPT-5)와 Claude Code(Anthropic)를 사용하였다. 저자들은 모든 참고문헌·정식화·코드·결과·해석을 검토·검증하였으며 내용에 대한 전적인 책임을 진다.

References

- [1] M. T. Assadi and M. Bagheri, “Differential evolution and population-based simulated annealing for truck scheduling problem in multiple door cross-docking systems,” *Computers & Industrial Engineering*, vol. 96, pp. 149–161, 2016.
- [2] P. Bodnar, R. de Koster, and K. Azadeh, “Scheduling trucks in a cross-dock with mixed service mode dock doors,” *Transportation Science*, vol. 51, no. 1, pp. 112–131, 2017.
- [3] N. Boysen and M. Flidner, “Cross dock scheduling: Classification, literature review and research agenda,” *Omega*, vol. 38, pp. 413–422, 2010.
- [4] C. M. Joo and B. S. Kim, “Scheduling compound trucks in multi-door cross-docking terminals,” *International Journal of Advanced Manufacturing Technology*, vol. 64, pp. 977–988, 2013.

- [5] D. Konur and M. M. Goliás, “Analysis of different approaches to cross-dock truck scheduling with truck arrival time uncertainty,” *Computers & Industrial Engineering*, vol. 65, no. 4, pp. 663–672, 2013.
- [6] D. Konur and M. M. Goliás, “Cost-stable truck scheduling at a cross-dock facility with unknown truck arrivals,” *Transportation Research Part E*, vol. 49, no. 1, pp. 71–91, 2013.
- [7] A.-L. Ladier and G. Alpan, “Robust cross-dock scheduling with time windows,” *Computers & Industrial Engineering*, vol. 99, pp. 16–28, 2016.
- [8] R. Larbi, G. Alpan, P. Baptiste, and B. Penz, “Scheduling cross docking operations under full, partial and no information on inbound arrivals,” *Computers & Operations Research*, vol. 38, no. 6, pp. 889–900, 2011.
- [9] Y. Li, M. Mohammadi, X. Zhang, Y. Lan, and W. van Jaarsveld, “Integrated trucks assignment and scheduling problem with mixed service mode docks: A Q-learning based adaptive large neighborhood search algorithm,” *European Journal of Operational Research*, vol. 333, no. 1, pp. 117–137, 2026.
- [10] D. Molavi, A. Shahmardan, and M. S. Sajadieh, “Truck scheduling in a cross docking systems with fixed due dates and shipment sorting,” *Computers & Industrial Engineering*, vol. 117, pp. 29–40, 2018.
- [11] A. Rijal, M. Bijvank, and R. de Koster, “Integrated scheduling and assignment of trucks at unit-load cross-dock terminals with mixed service mode dock doors,” *European Journal of Operational Research*, vol. 278, no. 3, pp. 752–771, 2019.
- [12] A. Shahmardan and M. S. Sajadieh, “Truck scheduling in a multi-door cross-docking center with partial unloading,” *Computers & Industrial Engineering*, vol. 139, Art. 106134, 2020.
- [13] J. Van Belle, P. Valckenaers, and D. Cattrysse, “Cross-docking: State of the art,” *Omega*, vol. 40, no. 6, pp. 827–846, 2012.
- [14] J. Van Belle, P. Valckenaers, G. Vanden Berghe, and D. Cattrysse, “A tabu search approach to the truck scheduling problem with multiple docks and time windows,” *Computers & Industrial Engineering*, vol. 66, no. 4, pp. 818–826, 2013.
- [15] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, pp. 279–292, 1992.
- [16] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [17] X. Xi, C. Liu, Y. Wang, and L. H. Lee, “Two-stage conflict robust optimization models for cross-dock truck scheduling problem under uncertainty,” *Transportation Research Part E*, vol. 144, Art. 102123, 2020.

Table 2: Test results over 20 instances per experimental condition (20 instances $\times$ 5 replications). Δ_{bo} is the mean gap to the best observed solution and time is the mean per run. The vs CP-SAT column is computed only on the subset for which CP-SAT returned a reference (5 instances per S condition, 2 for M-none); optimality was proven only for S-none. On that subset the mean CP-SAT runtime is 76 s (S) and 237 s (M-none).

실험 조건	방법	평균목적값 $\pm$ 표준편차	$\Delta_{bo}(\%)$	vs CP-SAT(%)	시간(초)
S-none	VAA	1497.5 $\pm$ 400.3	6.60	+6.49	0.00
	Paper-SA-RL5	1423.3 $\pm$ 383.6	1.03	+1.21	0.15
	GILS-uniform	1411.8 $\pm$ 382.7	0.16	+0.21	0.08
	GILS-tabular	1412.9 $\pm$ 381.3	0.28	+0.33	0.11
	GILS-DQN	1416.1 $\pm$ 382.4	0.51	+0.75	0.23
S-medium	VAA	5447.5 $\pm$ 1468.9	3.89	+3.33	0.00
	GILS-uniform	5255.7 $\pm$ 1413.3	0.08	+0.11	0.08
	GILS-tabular	5259.1 $\pm$ 1412.3	0.15	+0.18	0.11
	GILS-DQN	5270.8 $\pm$ 1415.3	0.38	+0.55	0.22
S-tight	VAA	9276.8 $\pm$ 2387.6	2.90	+2.17	0.00
	GILS-uniform	9010.8 $\pm$ 2206.1	0.09	+0.21	0.09
	GILS-tabular	9011.9 $\pm$ 2206.7	0.11	+0.17	0.11
	GILS-DQN	9022.0 $\pm$ 2205.3	0.23	+0.34	0.22
M-none	VAA	3030.9 $\pm$ 903.6	3.73	+4.80	0.00
	Paper-SA-RL5	2976.4 $\pm$ 864.4	1.86	+0.99	0.55
	GILS-uniform	2928.9 $\pm$ 842.6	0.29	+0.13	0.43
	GILS-tabular	2932.1 $\pm$ 846.3	0.37	+0.11	0.52
	GILS-DQN	2932.5 $\pm$ 845.5	0.40	+0.19	0.73
M-medium	VAA	23025.5 $\pm$ 4075.4	2.56	—	0.00
	GILS-uniform	22526.4 $\pm$ 3960.5	0.29	—	0.44
	GILS-tabular	22515.5 $\pm$ 3964.0	0.24	—	0.55
	GILS-DQN	22540.3 $\pm$ 3956.4	0.36	—	0.75
M-tight	VAA	38903.7 $\pm$ 11283.5	1.25	—	0.01
	GILS-uniform	38528.0 $\pm$ 11078.0	0.19	—	0.48
	GILS-tabular	38512.1 $\pm$ 11059.5	0.16	—	0.60
	GILS-DQN	38543.6 $\pm$ 11077.6	0.24	—	0.81
L-none	VAA	5492.3 $\pm$ 1482.2	2.21	—	0.02
	Paper-SA-RL5	5479.7 $\pm$ 1449.5	1.97	—	1.92
	GILS-uniform	5388.4 $\pm$ 1427.3	0.27	—	2.16
	GILS-tabular	5385.5 $\pm$ 1424.6	0.22	—	2.54
	GILS-DQN	5386.1 $\pm$ 1425.2	0.23	—	2.99
L-medium	VAA	55611.1 $\pm$ 18087.3	1.52	—	0.02
	GILS-uniform	54846.4 $\pm$ 17523.5	0.10	—	2.10
	GILS-tabular	54833.4 $\pm$ 17526.8	0.07	—	2.52
	GILS-DQN	54848.8 $\pm$ 17522.9	0.11	—	2.85
L-tight	VAA	119671.9 $\pm$ 32475.3	0.77	—	0.02
	GILS-uniform	118901.4 $\pm$ 31781.1	0.08	—	2.01
	GILS-tabular	118865.6 $\pm$ 31788.4	0.05	—	2.51
	GILS-DQN	118888.8 $\pm$ 31789.1	0.07	—	2.76